UTN · TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN
BASES DE DATOS II

Trabajo práctico - Unidad 6

Mantenimiento, respaldo y recuperación de Food Store

Integrantes
Gonzalo Sanchez
Roman Avila
Mailen Ortiz
Leo Fachinelli
Maximiliano Mendez
Gabriel D'Anna

Fecha: 28 de octubre de 2026 (UTC-03).

Alcance: resolución con scripts y prueba real sobre el esquema y los datos de ejemplo del repositorio del equipo, reconstruidos en PostgreSQL 18.6. Las mediciones corresponden a la semilla, no a la base de 251 MB documentada en el repositorio.

1. SPEC_U6.md - Plan de mantenimiento

Objetivo. Automatizar respaldo custom, monitoreo, mantenimiento y prueba de recuperación. Origen: foodstore. Destino exclusivo: foodstore_restore_test. Motor requerido: PostgreSQL 16+; cliente y servidor probados: 18.6.

Respaldo diario, 00:30. Ejecutar backup_foodstore.ps1. Éxito: salida 0, archivo fechado no vacío, catálogo legible por pg_restore --list y log de inicio, fin y tamaño. Se guarda baseline y fecha del inicio junto al dump.

Salud diaria, 08:00. Ejecutar health_check.sql y guardar salida. Éxito: cuatro consultas sin error, con tamaño, conexiones, tablas y consultas activas. Una lista vacía de consultas activas es válida.

Mantenimiento semanal, lunes. Ejecutar maintenance.sql y guardar salida. Éxito: revisar autovacuum y estadísticas; registrar acciones justificadas por métricas.

Restauración semanal, viernes. Ejecutar restore_test.ps1. Éxito: conteos idénticos al baseline, controles inválidos en cero y tiempos registrados. Si el destino existe, detenerse sin borrarlo.

Seguridad. Nunca eliminar ni reemplazar foodstore. Sin contraseñas en scripts ni entrega; usar pgpass.conf privado en el entorno habitual. El laboratorio descartable usó trust solo en 127.0.0.1:55436, sin cambiar la autenticación del servicio instalado. Scripts de escritura ensayados con BEGIN/ROLLBACK; respaldo antes de cambios estructurales.

Consistencia. Suspender escrituras desde la captura del baseline hasta terminar pg_dump. Cada dump conserva su propio baseline. Con concurrencia, compartir un snapshot entre conteos y respaldo.

Objetivos: RPO ≤ 24 h; RTO ≤ 10 min. RPO observado: inicio del simulacro menos inicio del respaldo. RTO: desde inmediatamente antes de createdb hasta validaciones exitosas.

Adaptación: el proyecto contiene usuario, relacionada con pedido.usuario_id; se valida usuario en lugar de cliente. No se crea una tabla ficticia.

2. Respaldo automatizable

backup_foodstore.ps1. Requiere herramientas PostgreSQL en PATH. La copia temporal .partial se renombra solo tras exportación y lectura de catálogo correctas. Los fallos devuelven código 1.

```
param([string]$Server='localhost', [int]$Port=5432,
      [string]$DbUser='postgres')
$ErrorActionPreference='Stop'
$c=@('-h',$Server,'-p',"$Port",'-U',$DbUser,'-w')
$stamp=Get-Date -Format 'yyyyMMdd_HHmmss_fff'
$root=Join-Path $PSScriptRoot 'backups'
$logs=Join-Path $PSScriptRoot 'logs'
New-Item -ItemType Directory -Force $root,$logs | Out-Null
$dump=Join-Path $root "foodstore_$stamp.dump"
$log=Join-Path $logs "backup_$stamp.log"
$start=Get-Date
try {
  "INICIO $($start.ToString('o'))" | Set-Content $log
  # Sin escrituras desde el conteo hasta terminar pg_dump.
  $baseline=& psql @c -X -A -t -v ON_ERROR_STOP=1 -d foodstore `
    -f (Join-Path $PSScriptRoot 'validate.sql') 2>> $log
  if ($LASTEXITCODE -ne 0) { throw 'Fallo baseline' }
  $baseline | Set-Content "$dump.baseline.txt"
  & pg_dump @c -d foodstore -Fc -f "$dump.partial" 2>> $log
  if ($LASTEXITCODE -ne 0) { throw 'Fallo pg_dump' }
  if ((Get-Item "$dump.partial").Length -eq 0) { throw 'Dump vacio' }
  & pg_restore --list "$dump.partial" >> $log 2>&1
  if ($LASTEXITCODE -ne 0) { throw 'Archivo ilegible' }
  Move-Item -LiteralPath "$dump.partial" -Destination $dump
  $start.ToString('o') | Set-Content "$dump.started.txt"
  "OK $(Get-Date -Format o) bytes=$((Get-Item $dump).Length)" |
    Add-Content $log
  exit 0
} catch {
  "ERROR: $_" | Add-Content $log
  exit 1
}
```

3. Monitoreo - health_check.sql

```
SELECT current_database() AS base,
       pg_size_pretty(pg_database_size(current_database())) AS tamano;
SELECT numbackends, xact_commit, xact_rollback
FROM pg_stat_database WHERE datname = current_database();
SELECT schemaname, relname, n_live_tup, n_dead_tup,
       last_autovacuum, last_autoanalyze, last_analyze
FROM pg_stat_user_tables ORDER BY n_dead_tup DESC;
SELECT pid, state, now() - query_start AS duracion, query
FROM pg_stat_activity
WHERE datname = current_database() AND state <> 'idle'
  AND pid <> pg_backend_pid();
```

Captura real: foodstore ocupaba 8486 kB; una conexión, 87 transacciones confirmadas y 5 revertidas. No había consultas activas ajenas a la sesión de control. Los contadores son acumulados de la instancia de laboratorio.

Tabla	Vivas estimadas	Muertas estimadas
categoria	4	8
producto	10	20
usuario	5	5
pedido	5	15
detalle_pedido	10	10

Las tuplas de pg_stat_user_tables son estimaciones. COUNT(*) se usa para validación exacta. Las tuplas muertas provienen de ensayos revertidos y actualización de totales. last_autovacuum y last_autoanalyze estaban sin valor en esta instancia recién creada; esto no demuestra una falla.

Ejecutar con psql -X -w -d foodstore -v ON_ERROR_STOP=1 -f health_check.sql y guardar la salida en logs/health_FECHA.log, comprobando el código de salida.

3. Mantenimiento - maintenance.sql

```
-- Ejecutar con psql -X -v ON_ERROR_STOP=1 -f maintenance.sql
-- Fuera de BEGIN: VACUUM no admite bloque transaccional.
SHOW autovacuum;
SELECT relname, n_live_tup, n_dead_tup, n_mod_since_analyze,
       last_autovacuum, last_autoanalyze, last_vacuum, last_analyze
FROM pg_stat_user_tables ORDER BY n_dead_tup DESC;
-- Criterio local de laboratorio: limpiar si hay >= 50 muertas
-- y superan 10% de las vivas. Revisar semanalmente el umbral.
SELECT format('VACUUM (ANALYZE) %I.%I;', schemaname, relname)
FROM pg_stat_user_tables
WHERE n_dead_tup >= 50
  AND n_dead_tup > 0.10 * greatest(n_live_tup, 1)
\gexec
-- Estadisticas sin analizar o cambios >= 10%: ANALYZE puntual.
SELECT format('ANALYZE %I.%I;', schemaname, relname)
FROM pg_stat_user_tables
WHERE (last_analyze IS NULL AND last_autoanalyze IS NULL)
   OR n_mod_since_analyze > 0.10 * greatest(n_live_tup, 1)
\gexec
```

Decisión observada: autovacuum estaba en on. Las cinco tablas tenían modificaciones desde el último análisis superiores al 10% de las filas vivas, por lo que se ejecutaron cinco ANALYZE. Ninguna alcanzó 50 tuplas muertas: no se ejecutó VACUUM manual.

Estos umbrales son decisiones de laboratorio, no valores predeterminados del motor. ANALYZE actualiza las estadísticas del planificador. VACUUM normal permite reutilizar espacio; no se usa VACUUM FULL como tarea periódica.

Ejecutar el mantenimiento fuera de BEGIN: VACUUM no admite una transacción explícita. Las escrituras de prueba sí se ensayaron con BEGIN/ROLLBACK. Aclaración del protocolo: pg_dump es un respaldo lógico; PostgreSQL permite revertir muchas operaciones DDL, aunque CREATE DATABASE es una excepción.

4. Simulacro - restore_test.ps1

```
param([string]$Server='localhost', [int]$Port=5432,
      [string]$DbUser='postgres')
$ErrorActionPreference='Stop'
$c=@('-h',$Server,'-p',"$Port",'-U',$DbUser,'-w')
$logs=Join-Path $PSScriptRoot 'logs'
New-Item -ItemType Directory -Force $logs | Out-Null
$log=Join-Path $logs ('restore_'+(Get-Date -Format yyyyMMdd_HHmmss)+'.log')
try {
  $dump=Get-ChildItem (Join-Path $PSScriptRoot 'backups') `
    -Filter 'foodstore_*.dump' | Sort-Object Name -Descending |
    Select-Object -First 1
  if (!$dump) { throw 'No hay respaldo terminado' }
  $expected=Get-Content "$($dump.FullName).baseline.txt"
  $backupStart=[DateTimeOffset]::Parse(
    (Get-Content "$($dump.FullName).started.txt"))
  $incident=[DateTimeOffset]::Now
  $watch=[Diagnostics.Stopwatch]::StartNew()
  # Falla si la base ya existe. Nunca elimina bases.
  & createdb @c -T template0 foodstore_restore_test 2>> $log
  if ($LASTEXITCODE -ne 0) { throw 'Fallo createdb' }
  & pg_restore @c --exit-on-error --no-owner --no-privileges `
    -d foodstore_restore_test $dump.FullName 2>> $log
  if ($LASTEXITCODE -ne 0) { throw 'Fallo pg_restore' }
  $actual=@(& psql @c -X -A -t -v ON_ERROR_STOP=1 `
    -d foodstore_restore_test `
    -f (Join-Path $PSScriptRoot 'validate.sql') 2>> $log)
  if ($LASTEXITCODE -ne 0) { throw 'Fallo validacion SQL' }
  if (@(Compare-Object $expected $actual).Count) {
    throw 'Conteos o controles diferentes al baseline'
  }
  if ($actual[-1] -ne '0' -or $actual[-2] -ne '0') {
    throw 'Hay filas invalidas'
  }
  $watch.Stop()
  $actual | Add-Content $log
  "BACKUP=$($dump.Name) INICIO=$backupStart" | Add-Content $log
  "RTO_s=$($watch.Elapsed.TotalSeconds)" | Add-Content $log
  "RPO_s=$(($incident-$backupStart).TotalSeconds)" | Add-Content $log
  'OK' | Add-Content $log
  exit 0
} catch {
  "ERROR: $_" | Add-Content $log
  exit 1
}
```

4. Validaciones y evidencias

```
SELECT 'categoria' AS tabla, count(*) AS filas FROM public.categoria
UNION ALL SELECT 'usuario', count(*) FROM public.usuario
UNION ALL SELECT 'producto', count(*) FROM public.producto
UNION ALL SELECT 'pedido', count(*) FROM public.pedido
UNION ALL SELECT 'detalle_pedido', count(*) FROM public.detalle_pedido
ORDER BY tabla;
SELECT count(*) AS productos_invalidos FROM public.producto
WHERE precio < 0 OR stock < 0;
SELECT count(*) AS detalles_invalidos FROM public.detalle_pedido
WHERE cantidad <= 0 OR precio_unitario < 0;
```

Tabla	Antes del dump	Restaurada
categoria	4	4
usuario	5	5
producto	10	10
pedido	5	5
detalle_pedido	10	10

Controles: productos_invalidos = 0 y detalles_invalidos = 0. Los conteos coinciden exactamente. pg_restore devolvió 0 y restauró sin propietarios ni privilegios del origen.

Respaldo final: foodstore_20261028_203928_224.dump, 16.451 bytes. Catálogo legible y log terminado en OK. Se eligió la última copia completa; la primera copia de desarrollo no se utilizó para la recuperación.

Pruebas negativas: puerto incorrecto 55437: backup devolvió 1; destino ya existente: restore devolvió 1 sin eliminarlo. El intento fallido posterior no reemplaza el resultado exitoso del simulacro.

5. Informe post-incidente

Fecha y entorno. 28/10/2026, PostgreSQL 18.6 en 127.0.0.1:55436. Base foodstore reconstruida con schema.sql y data.sql del equipo. Restauración en foodstore_restore_test. No se usó la base masiva ni se modificó el servicio instalado.

Respaldo: foodstore_20261028_203928_224.dump. Inicio: 28/10/2026 20:39:28 (UTC-03). Formato custom, 16.451 bytes y catálogo validado. Sin escrituras durante baseline y copia.

Indicador	Previsto	Observado
RTO	≤ 600 s	0,8626518 s
RPO (antigüedad)	≤ 86.400 s	1,2526273 s

RTO incluye creación, restauración y validaciones. RPO se calculó como inicio del simulacro menos inicio del respaldo: es la antigüedad conservadora de la copia, no un conteo de transacciones perdidas. No hubo escrituras durante la ventana. Los tiempos de esta semilla no demuestran el rendimiento de una base masiva.

Validación: categoria 4, usuario 5, producto 10, pedido 5, detalle_pedido 10. Iguales antes y después. Ambos controles lógicos devolvieron cero. Resultado general: OK.

Problema real: BEGIN/ROLLBACK consumió IDs de las secuencias; la carga siguiente falló por categoria_id=1 inexistente. En el entorno vacío se respaldó el estado y se reiniciaron las cinco secuencias a 1 antes de repetir. También se corrigió la correlación ambigua del UPDATE de totales con alias explícitos p.id y d.pedido_id.

Mejora: repetir con la carga masiva del equipo y snapshot compartido si hay concurrencia; conservar copia externa y alertar ante fallos diarios. La frecuencia por sí sola no garantiza el RPO.

Asistencia: scripts y redacción asistidos con Codex. No se usó OpenCode en esta ejecución. Se verificaron códigos de salida, catálogo, restauración real, conteos y fallos controlados. El equipo debe hacer su revisión humana antes de entregar; no se atribuye una revisión que no haya ocurrido.

6. Reproducibilidad y fuentes

Repositorio: https://github.com/Gonza-12-Sanchez/TPs_BD_FoodStore
Commit: f9bac19a2cba54d8d7ecc8cdcc8d68c6685826c6.

Se omitió la sentencia create database copia_food_store al reconstruir el esquema. En data.sql se corrigió el UPDATE a: UPDATE pedido p SET total = (SELECT SUM(d.subtotal) FROM detalle_pedido d WHERE d.pedido_id = p.id) WHERE p.id IN (1,2,3,4,5). Se ensayó con ROLLBACK antes de confirmar, reiniciando secuencias de la base vacía tras el ensayo.

No se cargaron volumen, índices adicionales ni vistas de TP5. El baseline de 251 MB del repositorio pertenece a otra ejecución y no se presenta como evidencia propia. Se respaldó la base vacía antes de crear el esquema y antes de reiniciar secuencias. El servidor aislado se detuvo al finalizar.

Orden de ejecución: configurar PATH y acceso privado; preparar foodstore; ejecutar salud y mantenimiento; detener escrituras; ejecutar respaldo; reanudar escrituras; ejecutar restauración. Conservar logs y baseline. Para repetir, usar otro laboratorio o revisar manualmente el destino descartable: el script nunca lo elimina automáticamente.

Los scripts completos están incluidos en este documento para la entrega única en PDF. Las mediciones reflejan la ejecución de esos scripts en el laboratorio descrito.

Referencias técnicas consultadas

pg_dump: copia lógica y formato custom
https://www.postgresql.org/docs/16/app-pgdump.html

pg_restore: restauración y errores
https://www.postgresql.org/docs/16/app-pgrestore.html

VACUUM y ANALYZE
https://www.postgresql.org/docs/16/routine-vacuuming.html

Consigna: TPUnidad_6_Mantenimiento_FoodStore.pdf, páginas 2 a 5. Se resolvieron las cinco partes y se adaptó cliente a usuario según el esquema real.